



Introduction to `riscv-unified-db` (UDB)

Derek Hower, Qualcomm

What is UDB?

A few things, actually:

- A **single source of truth** with a **consistent, validated interface**
 - Why? : RVI data hard to find, lot of work is being duplicated
- A **set of tools** using the database
 - Specification validator (schema check)
 - Documentation generators
 - ISS generator
 - QEMU generator
 - ...

RISC-V is **modular** and **extensible** →
RVI artifacts should be **generated**

Where is UDB?

GitHub (owned by RVI, BSD-3 license):

- <https://github.com/riscv-software-src/riscv-unified-db>
- **Community help welcome**: the more the merrier!

Real uses:

- [Xqci: Qualcomm Microcontroller Extension](#)
- [CSC Certification Requirement Document for MC-1](#)

Proof of concept uses:

- [RISC-V ISA manual, with appendices](#)
- [RVA Profile Family, self-contained, with comparisons](#)
- [Configuration-specific architecture documentation](#)

Motivation



Existing issues in RVI

- **Information is difficult to find**

- What is Ziccif?, Where is version 2.0 of the A extension?, ...
- What is the assembly format of an instruction?

- **Versioning is inconsistent**

- Extensions have versions, manuals have versions, neither follow predictable pattern

- **Specification layout is inconsistent**

- Written by different authors, companies (great!)
- Extra mental energy for readers (not great)

- **Duplication**

- Who or what owns the ratified opcodes? (Sail?, riscv-opcodes?)

- **Specs are not machine readable**

- Comparison is hard (What's different between RVA/RVB?)
- Unnecessary busy work (why do we hand-write disassemblers, wavedrom diagrams, ...?)

Current “sources of truth”

riscv-opcodes

riscv-config

Sail model

Spike

riscv-isa-manual

Extension documents on GitHub, Google Drive

Profile documents

Binutils (assembly mnemonics)

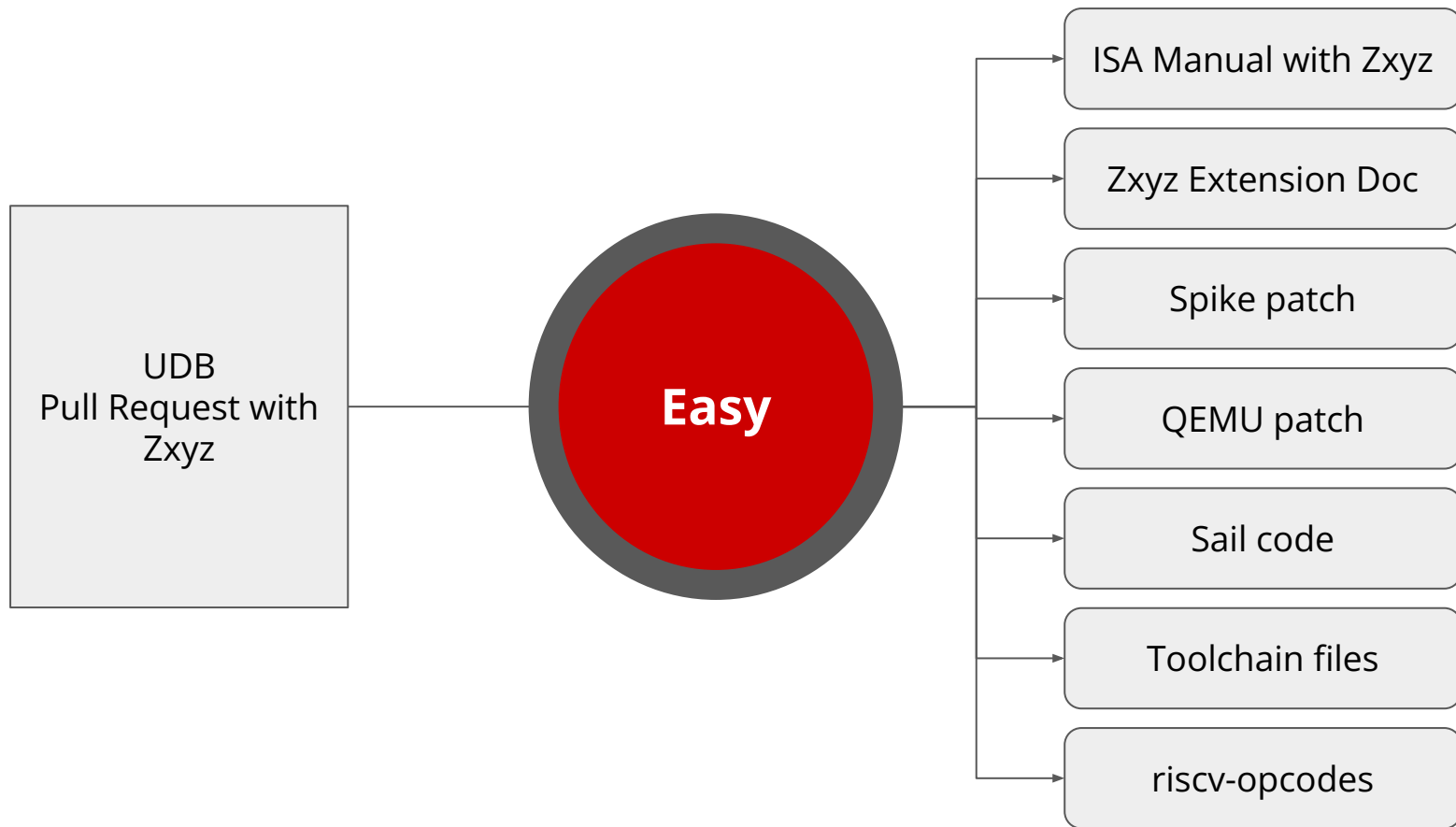
Riscv assembly guide

RVI JIRA

...



What if...



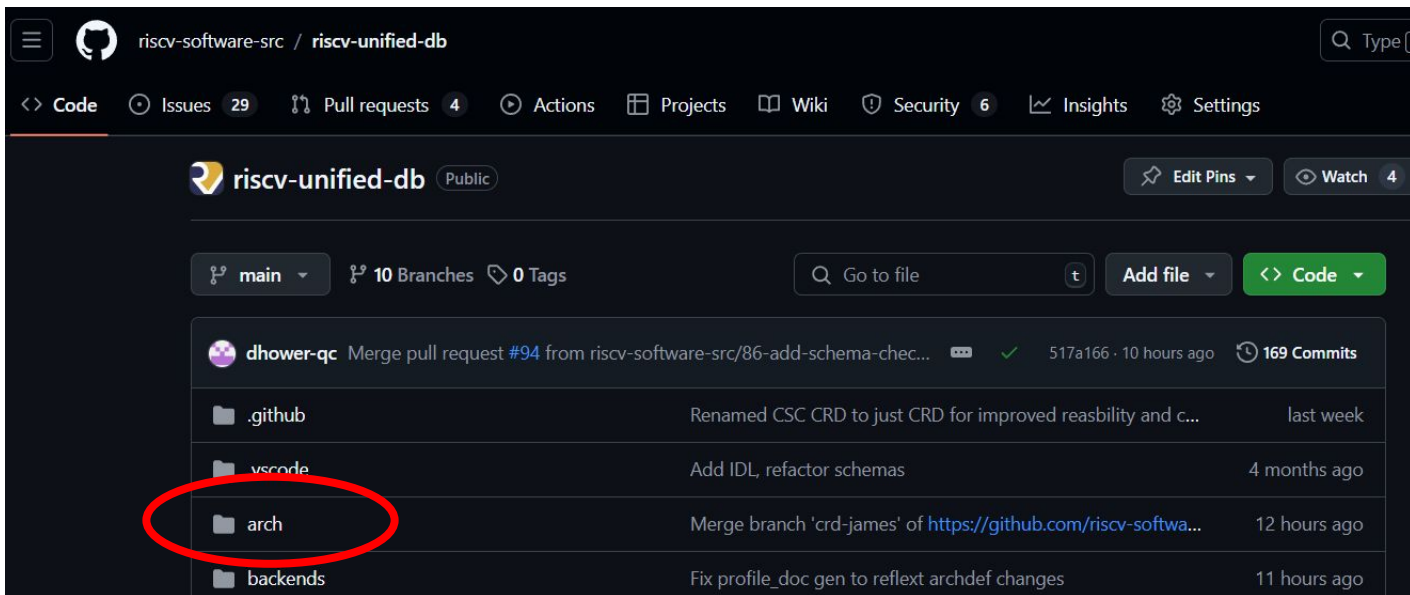
Database Format



Database format

All data is stored in [YAML](#) files

- Both machine and human readable (in theory)
- Not tied to any specific tool/language (in theory)



Database format

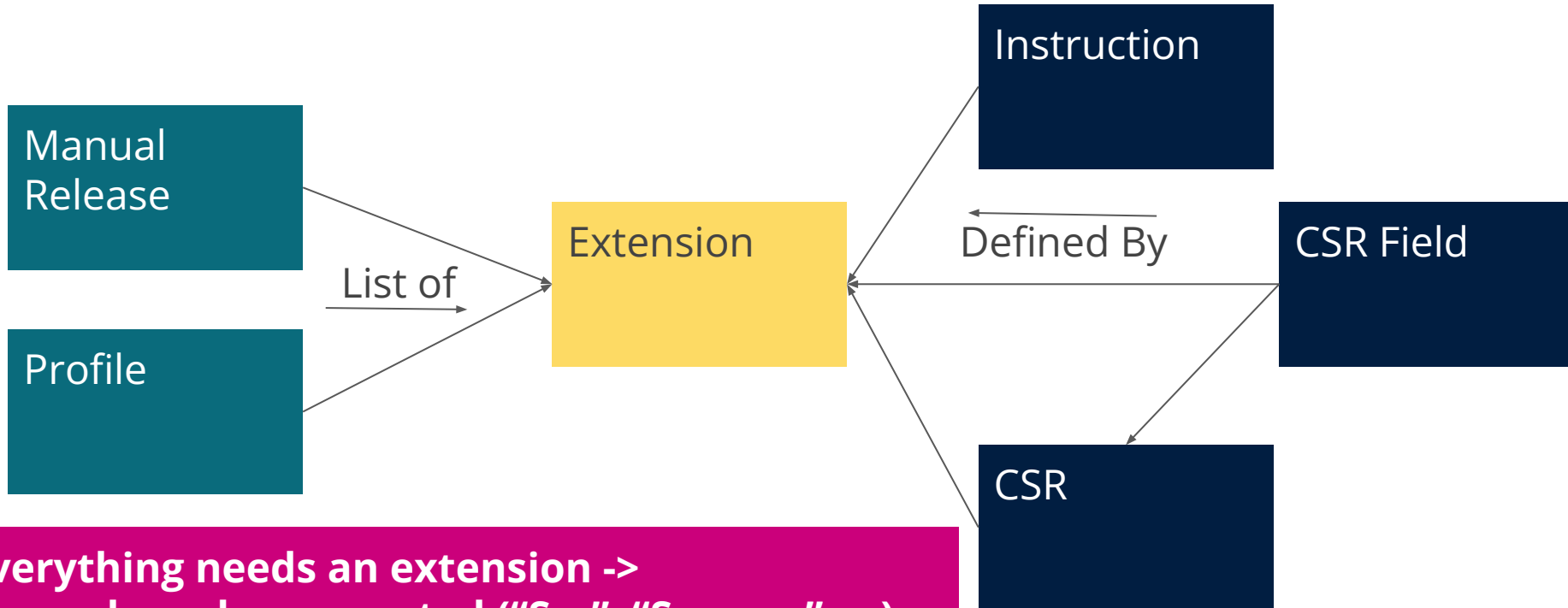
One YAML file per instruction/CSR/extension/profile/manual. Combination of:

- Metadata (authors, spec state, etc.)
- Normative specification prose
- Executable specs (unofficial, in IDL)
- Semantic version information
 - Sometimes inferred when no version is given by specs (e.g., ratified defaults to 1.0)

Goal to capture entire spec, *including all options*

- Most options have no explicit name; `riscv-unified-db` gives them one
- Work in progress to compare with `riscv-config`, Imperas/Synopsys

Basic Entity Relationship



**Everything needs an extension ->
some have been created ("Sm", "Smpmp", ...)**

Instructions

add:

long_name: Integer add

description: |

Add the value in rs1 to rs2, and store the result in rd.

Any overflow is thrown away.

definedBy: I

assembly: xd, xs1, xs2

encoding:

match: 0000000-----000-----0110011

variables:

- **name:** rs2

location: 24-20

- **name:** rs1

location: 19-15

- **name:** rd

location: 11-7

access: # ...

data_independent_timing: true

operation(): X[rd] = X[rs1] + X[rs2];

Instructions

add:

long_name: Integer add

description: |

Add the value in rs1 to rs2, and store the result in rd.

Any overflow is thrown away.

definedBy: I

assembly: xd, xs1, xs2

encoding:

match: 0000000-----000-----0110011

variables:

- **name:** rs2
location: 24-20
- **name:** rs1
location: 19-15
- **name:** rd
location: 11-7

access: # ...

data_independent_timing: true

operation(): X[rd] = X[rs1] + X[rs2];

Every instruction must belong to at least one extension

Encoding (same information from riscv-opcodes)

Formal operation, in IDL

CSRs

mstatus:

long_name: Machine Status

address: 0x300

priv_mode: M

length: MXLEN

description: The mstatus register tracks and controls the hart's current operating state.

definedBy: Sm

fields:

...

CSRs

mstatus:

long_name: Machine Status

address: 0x300

priv_mode: M

length: MXLEN

description: The mstatus register tracks and controls the hart's current operating state.

definedBy: Sm

fields:

...

Address (CSR* instructions)

Also has “virtual_address” for Hypervisor
and “indirect_address” for Zicsrind

Every CSR must be defined by at least one extension

Fields inherit CSR definedBy, but can override when field
introduced by different extension

CSRs

```
# ...
fields:
  # ...
  MBE:
    location: 37
    base: 64
    description: |
      *M-mode Big Endian*

      Controls the endianness of data M-mode (0 = little, 1 = big).
      Instructions are always little endian, regardless of the data setting.

      # ...
    type(): |
      return (M_MODE_ENDIANESS == "dynamic") ? CsrFieldType::RW : CsrFieldType::RO;

  # if endianness is mutable, MBE comes out of reset in little-endian mode
  reset_value(): |
    return (M_MODE_ENDIANESS == "big") ? 1 : 0;
```

CSRs

```
# ...
fields:
  # ...
  MBE:
    location: 37
    base: 64
    description: |
      *M-mode Big Endian*
```

← Only defined in RV64

Controls the endianness of data M-mode (0 = little, 1 = big).
Instructions are always little endian, regardless of the data setting.

... Type and reset value depend on parameter; expressed in IDL

```
type(): |
  return (M_MODE_ENDIANESS == "dynamic") ? CsrFieldType::RW : CsrFieldType::RO;
```

if endianness is mutable, MBE comes out of reset in little-endian mode

```
reset_value(): |
  return (M_MODE_ENDIANESS == "big") ? 1 : 0;
```

More!

Also understands:

- Extensions
- Profiles / profile families (RVA)
- Certification Requirements Documents (CRDs, from CSC)
- Manual releases

Example Artifacts (Demo Time!)



Generated ISA Manual

- Creates **versioned** views of RISC-V unpriv, priv manuals
- Includes **appendix** of all instructions, CSRs, extensions, parameters
 - Consistent layout, information across all extensions
 - Generated encoding diagrams
- Linked (clickable) executable specification
- Highlights **differences** between RV32/RV64
- Highlights **runtime-dependent functionality**
- (unratified) **Context-specific prose**
- **Names unnamed options**

Implementation Documentation

Prunes the displayed spec for a configuration, including:

- Extension/Instruction/CSR list
- Executable code (i.e., dead code removal)

Layout **consistent** with ISA manual (fewer surprises)

Profile Generator

Self-contained RVA profile specification

- Profile-specific extension/instruction/CSR lists

Generated **comparison among profile releases** (RVA20/22/..)

- Could also compare profile families (RVA/RVB)

CRD

Certification Requirements Document for CSC

- Explains, **in exacting detail**, what CSC will certify

Database case study: More work than Google Doc, but **better product**

- ensures that **everything is specified**

Extension Document

Could be used by TG to create **ratification-ready document**

No extra work to integrate in manual

ISS/QEMU

Generated C++ hart model - Coming soon!

- Exists in Qualcomm, have approval to open source
- Can be *either* specific to an implementation, fully runtime configurable, or a mix of both (e.g., an RVA hart model)

QEMU TCG Generator

- **rev.ng Labs** can convert IDL executable spec to TCG
- Done for Xqci:

Outline

1. Brief QEMU intro
2. helper2tcg: from interpreter to JIT translator
3. Demo: helper2tcg on xqciu
4. Generating test cases with KLEE
5. What's next: using LLVM JIT in QEMU

Interacting with the database



Working with the tools

Everything set up to run using Singularity/Apptainer container

Driven by [rake](#) (Ruby Make)

The wrapper script `do` runs everything in the container

```
cd RISCV_UNIFIED_DB
```

```
./do --tasks      # show all tasks
```

```
./do --desc       # show more about all tasks
```

```
./do validate     # check schemas, IDL, etc.
```

```
./do gen:manual_html MANUAL_NAME=isa # generate all versions of the manual
```

Ruby model

riscv-unified-db includes Ruby classes to extract information from database

```
# all known extensions
archdef.extensions

# all known instructions from the 'B' extension
archdef.extension("B").instructions

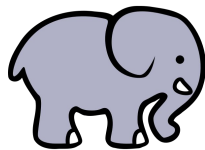
# all extensions defined in ISA manual version 20240411
archdef.manual("isa").version("20240411").extensions
```

API documented on GitHub:

ISA Description Language (IDL)



Why not Sail?



At the time riscv-unified-db started:

- RISC-V Sail Model was not widely used
- ... was incomplete
- ... was not tested
- ... was not familiar to anyone in Qualcomm

We had a choice:

- Learn Sail, *including how to create/modify a Sail compiler*
- Start from scratch, **learning from** our experience studying the Sail model

We chose the latter

(side note): the plan is to pull Sail snippets into the database

IDL Language Features

Unambiguous, strictly-typed description language (like Sail)

- Borrows most syntax/semantics from C/Verilog

Designed for RISC-V specification

- Understands configuration options (extensions, options)
- CSRs, X registers are built in to the language

Expects uncertainty:

- “UNPREDICTABLE”, “UNDEFINED_LEGAL” are valid values

Integrated with YAML data, e.g.:

- Type (RO, R/W, etc) of CSR field is specified in IDL when type is config-dependent

IDL Compiler Features

Symbolic execution

- E.g., to calculate all possible synchronous exceptions from an instruction

Compiles partials

- E.g., to understand a CSR's type

Dead code analysis

- Prunes source to make presentation configuration-dependent

IDL Example (div operation)

```
if (CSR[misa].M == 1'b0) {
    raise (ExceptionCode::IllegalInstruction, mode(), $encoding);
}

XReg src1 = X[rs1];
XReg src2 = X[rs2];

if (src2 == 0) {
    # division by zero. Since RISC-V does not have arithmetic exceptions, the result is
    # defined to be -1
    X[rd] = {XLEN{1'b1}};

} else if ((src1 == {1'b1, {XLEN-1{1'b0}}}) && (src2 == {XLEN{1'b1}})) {
    # signed overflow. Since RISC-V does not have arithmetic exceptions, the result is
    # defined to be the most negative number ( $-2^{(XLEN-1)}$ )
    X[rd] = {1'b1, {XLEN-1{1'b0}}};

} else {
    # no special case, just divide
    X[rd] = $signed(src1) / $signed(src2);
}
```

IDL Example (div operation)

```
if (CSR[misa].M <= 1'b0) { CSR is a built-in
    raise (ExceptionCode::IllegalInstruction, mode(), $encoding);
}
```

```
XReg src1 = X[rs1];
XReg src2 = X[rs2];
```

X register file, including X0, is a built-in

```
if (src2 == 0) {
    # division by zero. Since RISC-V does not have arithmetic exceptions, the result is
    # defined to be -1
```

```
X[rd] = {XLEN{1'b1}};
```

```
} else if ((src1 == {1'b1, {XLEN-1{1'b0}}}) && (src2 == {XLEN{1'b1}})) {
    # signed overflow. Since RISC-V does not have arithmetic exceptions, the result is
    # defined to be the most negative number (-2^(XLEN-1))
    X[rd] = {1'b1, {XLEN-1{1'b0}}};
```

```
} else {
    # just divide, throw away remainder
    X[rd] = $signed(src1) / $signed(src2);
}
```

Mostly C, but
borrow a few things
from Verilog

Looking Forward



Status of major extensions

Done, with IDL (lightly tested):

- I, M, A, C, B
- Sm, S, U,

Started:

- H (~65% done)
- F [native IDL] (~50% done)

Not started:

- V

Needs

Data input

- including extraction from other sources where possible (thanks Synposys!)

Validation

Feature improvements

Use!

- Active conversations, help on GitHub Issues/PRs
- Reach out to Derek Hower (dhower@qti.qualcomm.com) for a chat

Learning

Talk at NA summit Member Day (Monday, 10/21) with Paul Clarke (Ventana)

Deep dive “tutorial” sessions. I’m ready when you are.

GitHub repo actually (!!) has ok documentation (though it could be better organized)

Thanks!



Backup



What can UDB generate?

Generate entire priv/unpriv spec, *with appendix of instructions, CSRs, extensions*

Generate configuration-specific spec

- Irrelevant insts/CSR (fields)/extensions/executable specs pruned out
- With *overlays* for custom behavior

Generate documentation for *all versions of* an extension

Generate a C++ hart model from executable spec (coming soon)

- Either **fully generic** (all options set at runtime), or
- **config-specific** (some/all options set at compile time)

... and more

Extension

Everything in `riscv-unified-db` belongs to a specific extension version

- “Floating” specs are given names, e.g., “Sm” for mandatory privileged, “Smpmp” for PMP, etc.

Captures inter-extension relationships:

- Implies (e.g., B implies Zba, Zbb, Zbs)
- Requires (e.g., D requires F)
- Excludes (e.g., Zfinx excludes F)

Exhaustive list of parameters

- **The (often unnamed) options available to an implementation**

Semantically versioned

- Starting from “current” → e.g., A starts at version 2.2

Instruction

Belongs to one or more Extensions

Specifies:

- Mnemonic & assembly syntax
- Long name, description
- Encoding, including decode variables (analogous to `riscv-opcodes`)
- Formal executable specification (in IDL -)

CSR

One file per CSR; contains all fields, too

- Often, CSR will be affected by many extensions

CSR data:

- Extension relationships
- Width (RV32 and RV64, when different)
- Address, virtual address (for VS-mode), indirect address (for Scsrind)

Field data:

- Reset state
- Type (RO, RW, with/without hardware update)
- Behavior on read / behavior on write (to specify, e.g., WARL)